

Plantilla de documentación del proceso de diseño de soluciones y proyectos

Nombre completo

Adonis Eloy Roman Dinarte

IteraFlex: Diseño Ágil de Soluciones y Proyectos

El proceso IteraFlex es una herramienta para guiar el desarrollo de soluciones y proyectos y convertir nuestras ideas en realidad. IteraFlex facilita la empatía con el usuario final, el aprendizaje continuo de los errores y la resolución de problemas de forma creativa. Este método es iterativo, lo que significa que puede repetirse varias veces para encontrar la mejor solución posible.

En esta plantilla, completaremos la información necesaria para documentar el proceso de desarrollo de su proyecto mediante IteraFlex.



Enlace al proyecto:

<https://github.com/RomanD1995/entrega-proyecto-final-de-curso>

Paso 1: Definir el problema

En este paso se define el problema a resolver o proyecto a desarrollar de forma clara. También se identifican las limitaciones y los obstáculos a los que nos enfrentamos. Esto usualmente implica comunicarnos con empatía con las personas para las que se diseña. La idea es clarificar cuál es la necesidad o problema a resolver y abstraer lo esencial del problema a resolver para enfocarnos en ello.

Tomando en cuenta la definición anterior conteste las siguientes preguntas:

Pregunta A

¿Cuál es el objetivo del proyecto que va a realizar?

Evaluar el rendimiento operativo, los cuellos de botella y las tendencias de la rampa de alineado y tramado del taller durante un periodo de algunos meses. El objetivo es cuantificar la demanda del servicio, analizar la distribución de categorías de garantía según el estado de la suspensión (Apto 100% garantizado vs. Con daño/Sin garantía) y medir la venta cruzada de servicios complementarios (llantas nuevas, balanceos y pinchaduras) para respaldar la toma de decisiones estratégicas del negocio, así como calcular la densidad del servicio y la salud general de los vehículos de la flotilla de la zona.

Pregunta B

¿Qué componentes debe tener?

- *Modulo de lectura y limpieza de datos:*
acá vamos a cargar lo datos desde un archivo (CSV) extraído desde el archivo de reportes de la máquina de alineación Hunter, con manejo de acepciones y limpieza de los datos
- *Estructuras de datos:*
utiliza DataFrames de pandas, listas y diccionarios para organizar columnas, categorías, estados y servicios.
- *Motor de análisis:*
calcularemos los promedios de ocupación, frecuencia del servicio, tendencia por marcas o modelos mas recurrentes y vehículos no aptos por su estado mecánico, módulos de salud, capacidad operativa y servicios secundarios.
- *Menú de interacción:*
Generamos unos filtros por marcan, modelo, fecha o estatus de garantía ,así como escoger lo que deseamos analizar como carga y capacidad operativa de la rampa, salud vehicular o generarnos gráficos sobre estas métricas
- *Apartado de reportes y visualización: consolidado con un resumen grafico de la métricas encontradas en los datos y gráficos descriptivos*

Pregunta C

¿Cuáles son las preguntas clave que el análisis debe responder?

¿Qué porcentaje de vehículos viene al servicio en su estado optimo?
¿en qué porcentaje el alineado genera ventas con servicios cruzados (complementarios)?
¿cuál es el volumen mensual de alienados con una única rampa?
¿qué tanto influye el servicio a los carros defectuosos rechazados o sin garantía?

Pregunta D

¿Qué datos específicos son necesarios?

Identificación = id_boleta
Temporalidad = fecha_servicio
Vehículos = placa_vehiculo
Marca = marca_vehiculo
Modelo = estilo_vehiculo
Año = anio_vehiculo
Estatus de suspensión = (optimo, Daño_decide_reparar, Daño_Acepta_sin_garantia)
Garantía = (cat_a_garantizado, cat_b_reparacion, cat_c_sin_garantia)
Llantas nuevas = llantas
Balanceo = balanceo
Pinchadura = reparación
Desviación inicial = desviación
Alineación exitosa = alineado_exito

Paso 2: Investigar

En esta etapa podemos hablar con distintas personas, realizar búsquedas en línea para investigar qué proyectos o soluciones ya existen, o qué tecnologías podrían adaptarse a nuestras necesidades. También es importante indagar el contexto y las implicaciones del problema.

Investigar es hacer preguntas y buscar respuestas. Muchas veces las respuestas nos llevan a otras preguntas que nos hacen pensar sobre nuestro problema desde perspectivas nuevas. Ahora investigue en relación a la aplicación que va a crear para su proyecto final y complete las siguientes preguntas:

Pregunta C

Liste al menos dos fuentes en las que indagó sobre el contexto del proyecto a realizar.

1.	<i>Registros históricos de la alineadora Hunter PA130</i>
2.	<i>Entrevista con el personal técnico</i>
3.	<i>Muestreo de saturación de la rampa</i>
4.	<i>Entrevista con administrador del establecimiento</i>
5.	<i>Registro de cobro de los servicios secundarios</i>

Pregunta D

Comparta al menos una solución existente que aborde su necesidad.

1.	<i>Sistemas ERP/DMS Automotrices comerciales (con módulos integrados de gestión de taller) , ejemplo: Audatex , DealerSocket, Tullapp.</i>
2.	<i>Hojas de calculo con tablas dinámicas en Excel , (plantillas manuales para el registros de ingresos de vehículos y generación de gráficos)</i>
3.	<i>Software nativo de las alineadoras de última generación Hunter ,(sistemas modernos que exportan de forma nativa desde las alineadoras bases de datos relacionales (SQL.)</i>

Paso 3: Plantear una solución

Este es el momento de abrir la mente y generar ideas de posibles abordajes para nuestra solución o proyecto y desarrollar tantas soluciones como sea posible. Estas soluciones pueden ser simples o complejas. Recuerde que las ideas evolucionan, se mezclan unas con otras y ¡no existen malas ideas!

Después de imaginar todas las posibles soluciones es tiempo de considerar las necesidades, limitaciones e investigación de los pasos anteriores y comparar las ideas, para seleccionar una solución y elaborar un plan para seguir adelante.

Al seleccionar una posible solución es útil modularizarla (plantear una solución programada que está en procedimientos que se ejecutan de manera articulada) para comprender sus partes y cómo estas se pueden ejecutar de forma articulada. A la hora de crear un plan es útil formular algoritmos o instrucciones paso a paso para completar una tarea.

En el paso 4 la idea es clarificar cuál es la necesidad o problema a resolver y abstraer lo esencial del problema a resolver para enfocarnos en ello.

Tomando en cuenta la definición anterior conteste las siguientes preguntas:

Pregunta E

Escriba sus ideas de solución (mínimo 3 ideas)

1.	<i>Crear un script en Python que procese los datos de un archivo SCV y monitoree el conteo de las variables .</i>
2.	<i>Desarrollo de una aplicación interactiva que cargue el data base de manera automática</i>

3.	Implementar una base de datos SQLite vinculada a una interfaz grafica con TKinter para el registro en tiempo real de nuevos servicios
4.	
5.	

Pregunta F

De las ideas de solución (Pregunta A), seleccione la idea que va a desarrollar

La idea 2 : desarrolla una aplicación interactiva en Python orientada al análisis de datos con pandas ,que procese el archivo csv exportado desde la alineadora Hunter permitiendo la filtración dinámica por tipos de vehículo , evaluando índices de rechazo ,garantía y visualización de salud de la flotilla local .

Pregunta G

Modularice y liste los procedimientos necesarios para llegar a la solución seleccionada (mínimo 3 procedimientos)

1.	Modulo de carga e integración de datos; lectura de archivos registro_taller_csv usando pandas y verificando inconsistencias (manejo de nulos, por fecha y limpieza de texto .
2.	Módulo de procesamiento y análisis estadístico; motor de análisis calcula y cuantifica las tasas de rechazo, frecuencia por marca, tipo de vehículo, tiempo promedio en rampa y consolidación de servicios secundarios
3.	Modulo de filtro e interacción ; menú interactivo para el usuario consulte métricas por categoría (sedan ,4x4, suv, pick_up) o por periodo de tiempo
4.	Modulo de visualización y reportes ; creamos gráficos de barras y sectores con Matplotlib par visualizar la salud vehicular y la ocupación de la rampa además de un índice por marcas en el servicio indicando la población de las mismas en la flotilla local y cuales requieren ,as el servicio.
5.	

Pregunta H

Desarrolle un algoritmo (en lenguaje cotidiano o lenguaje natural) que describa paso a paso las instrucciones para conseguir la solución a la idea seleccionada. (mínimo 3 pasos)

1.	(Inicio): El sistema inicia y busca el archivo registros_taller.csv en el directorio de trabajo. Si no lo encuentra, despliega un mensaje de error y solicita la ruta correcta.
2.	(Limpieza y Estructuración): Convierte las columnas de fecha a formato datetime, asegura que tiempo_rampa_minutos sea numérico y rellena

	<i>o identifica las celdas vacías como servicios no realizados o rechazados. Se extrajeron estos datos de forma manual a un archivo Excel ya que la alineadora no posee sistema de exportación los cuales se van a convertir a un archivo CSV</i>
3.	<i>(Menú de Opciones): Muestra al usuario un menú en pantalla con las opciones: [1] Resumen General de Salud Vehicular (% Aprobados vs. Rechazados). [2] Ocupación y Tiempo Promedio en Rampa. [3] Análisis por Tipo de Vehículo y Marcas Recurrentes. [4] Generar Gráficos de Indicadores Clave. [5] Salir.</i>
4.	<i>(Procesamiento de Selección): Según la opción elegida por el usuario, aplica los filtros correspondientes sobre el DataFrame de Pandas y ejecuta los cálculos requeridos.</i>
5.	<i>(Presentación de Resultados): Imprime en pantalla las tablas de resumen o despliega la ventana con la gráfica generada por Matplotlib.</i>
6.	<i>(Bucle y Cierre): Pregunta al usuario si desea realizar otra consulta. Si selecciona salir, finaliza la ejecución de forma limpia.</i>

Pregunta 1

Cree un programa siguiendo el algoritmo y que dé respuesta al problema planteado. Luego de crearlo y depurarlo, agregue imágenes del código del programa en este espacio:


```
carga_de_datos.py X graficos_e_indicadores.py X capacidad_operativa.py X servicios_secundarios.py X LEEME ANTES DE ANALIZAR.txt X calculo_de_salud.py X e
app_analisis_de_servicios.py > carga_de_datos.py > cargar_y_limpiar_base_de_datos
1
2
3 CARGA Y LIMPIEZA DE DATOS
4
5
6 from pathlib import Path
7
8 import pandas as pd
9
10 NOMBRE_DEL_PROYECTO = "Sistema de Análisis de Salud Vehicular y Taller Hunter"
11 VERSION_DEL_SISTEMA = "3.0 Modular"
12 AUTOR_DEL_PROYECTO = "Adonis Eloy Roman Dinarte"
13 RUTA_EXCEL_INICIAL = "base_datos_taller.xlsx"
14 RUTA_CSV_PROCESADO = "registros_taller_procesado.csv"
15 CARPETA_DEL_PROGRAMA = Path(__file__).resolve().parent
16
17
18 # Prueba varias rutas posibles y devuelve la primera que corresponde a un archivo existente.
19 # Para hacerlo, revisa cada ruta con Path y se detiene cuando encuentra un archivo válido.
20 def _buscar_archivo(ruta_entrada):
21     """Busca el archivo indicado y, si no existe, el CSV procesado."""
22     ruta_solicitada = Path(ruta_entrada)
23     rutas_posibles = [ruta_solicitada, CARPETA_DEL_PROGRAMA / ruta_solicitada]
24
25     if ruta_solicitada.name == RUTA_EXCEL_INICIAL:
26         rutas_posibles.extend([
27             CARPETA_DEL_PROGRAMA.parent / ruta_solicitada,
28             CARPETA_DEL_PROGRAMA / RUTA_CSV_PROCESADO,
29             CARPETA_DEL_PROGRAMA.parent / RUTA_CSV_PROCESADO,
30         ])
31
32     for ruta in rutas_posibles:
33         if ruta.is_file():
34             return ruta
35
36     return None
37
38
39 # Busca el archivo, lo convierte en una tabla, corrige sus datos y guarda una copia limpia.
40 # Para hacerlo, lee Excel o CSV, revisa columnas, convierte tipos, elimina duplicados y guarda el resultado.
41 def cargar_y_limpiar_base_de_datos(ruta_entrada=RUTA_EXCEL_INICIAL):
42     ruta_final = _buscar_archivo(ruta_entrada)
43     if ruta_final is None:
44         print(f"\n[ERROR] No se encontró el archivo en '{ruta_entrada}'")
45         return None
46
47     try:
48         # Revisa la extensión para decidir si pandas debe usar read_excel o read_csv
```

```

carga_de_datos.py X graficos_e_indicadores.py X capacidad_operativa.py X servicios_secundarios.py X LEEME ANTES DE ANALIZAR.txt X calculo
app_analisis_de_servicios.py > graficos_e_indicadores.py > ...
1  """
2  =====
3  GRÁFICOS E INDICADORES (MATPLOTLIB)
4  =====
5  """
6  import matplotlib.pyplot as plt
7  import pandas as pd
8
9
10 # Revisa dos nombres posibles y devuelve el primero que exista en la tabla.
11 # Para hacerlo, recorre una lista de nombres y comprueba cada uno dentro de las columnas.
12 def obtener_columna_evaluacion(datos_flota):
13     for columna in ['alineacion_exitosa', 'estatus_suspension']:
14         if columna in datos_flota.columns:
15             return columna
16     return None
17
18
19 # Limpia el texto recibido y lo compara con listas de estados buenos, malos o desconocidos.
20 # Para hacerlo, quita espacios, pasa el valor a minúsculas y lo busca en las listas de estados.
21 def normalizar_estado(valor):
22     valor = str(valor).strip().lower()
23     if valor in ['si', 'aprobado', 'apto', 'optimo', 'bueno', 'saludable', 'ok', 'exitoso']:
24         return 'Saludable / en buen estado'
25     if valor in ['no', 'rechazado', 'dañado', 'defectuoso', 'malo', 'en mal estado', 'fallo', 'reparar']:
26         return 'En mal estado'
27     return 'No especificado'
28
29
30 # Agrupa los estados de la tabla y usa sus cantidades para crear un gráfico circular.
31 # Para hacerlo, cuenta los estados normalizados y entrega esas cantidades a plt.pie.
32 def graficar_salud_general(datos_flota):
33     """Pie chart para composición del estado general de la flota."""
34     # Busca la columna de evaluación y detiene el gráfico si la tabla no tiene ninguna.
35     columna = obtener_columna_evaluacion(datos_flota)
36     if columna is None:
37         print("[AVISO] No existe una columna de diagnóstico para graficar salud general.")
38         return False
39
40     serie = datos_flota[columna].fillna('no_especificado').apply(normalizar_estado)
41     conteo = serie.value_counts()
42     if conteo.empty:
43         print("[AVISO] No hay datos para construir este gráfico.")
44         return False
45
46     # Crea una porción y muestra su porcentaje para cada estado contado.
47     fig, ax = plt.subplots(figsize=(8, 6))
48     colores = ['#2ca02c', '#d62728', '#7f7f7f']

```



```

de_datos.py X graficos_e_indicadores.py X capacidad_operativa.py X servicios_secundarios.py X LEEME ANTES DE ANALIZAR.txt X ca
aj C:\Users\veloyr\OneDrive\Desktop\entrega proyecto
final\app_analisis_de_servicios.py\carga_de_datos.py
2 =====
3 EJECUTABLE PRINCIPAL - SISTEMA ALINEADORA HUNTER
4 =====
5 """
6 import carga_de_datos as carga
7 import capacidad_operativa as capacidad
8 import calculo_de_salud as salud
9 import servicios_secundarios as servicios
10 import graficos_e_indicadores as graficos
11
12 # Carga la tabla una vez y dirige cada respuesta del usuario al analisis correspondiente.
13 # Para hacerlo, lee la opcion escrita y llama a la funcion del modulo que corresponde.
14 def menu_principal():
15     # Llama a la funcion de carga; si devuelve None, detiene el programa para evitar analisis sin datos.
16     datos_flota = carga.cargar_y_limpiar_base_de_datos()
17     if datos_flota is None:
18         print("\n[ERROR] No se pudo iniciar el programa.")
19         return
20
21     # Repite la impresion del menu y la lectura de opciones mientras el usuario no elija salir.
22     while True:
23         print("\n" + "="*65)
24         print(f" {carga.NOMBRE_DEL_PROYECTO.upper()}")
25         print(f" Versión: {carga.VERSION_DEL_SISTEMA} | Autor: {carga.AUTOR_DEL_PROYECTO}")
26         print("="*65)
27         print("[1] Capacidad Operativa y Carga de Rampa")
28         print("[2] Resumen General de Salud Vehicular (% Aprobados vs Rechazados)")
29         print("[3] Diagnóstico de Salud Cruzado por Marca")
30         print("[4] Análisis por Tipo de Vehículo y Servicios Secundarios")
31         print("[5] Generar Gráficos de Indicadores Clave")
32         print("[6] Salir")
33         print("="*65)
34
35         opcion = input("Seleccione una opción (1-6): ").strip()
36
37         # Compara el texto escrito con cada opcion y ejecuta la funcion del modulo seleccionado.
38         if opcion == '1':
39             capacidad.calcular_capacidad_y_desempeno_puesto(datos_flota)
40         elif opcion == '2':
41             salud.analizar_salud_vehicular_general(datos_flota)
42         elif opcion == '3':
43             salud.analizar_salud_por_marca(datos_flota)
44         elif opcion == '4':
45             servicios.analizar_servicios_secundarios_por_tipo(datos_flota)
46         elif opcion == '5':
47             graficos.generar_graficos_indicadores(datos_flota)
48         elif opcion == '6':

```

Rúbrica de Evaluación (25% de la nota del curso)

Avance del Proyecto 5%

Evaluación propuesta del Proyecto Final

Criterio	Excelente (3 puntos)	Bueno (2 puntos)	Insuficiente (1 punto)	No Presentado (0 puntos)	Puntos
----------	-------------------------	---------------------	---------------------------	-----------------------------	--------

Claridad y Objetivos	Explica claramente y con detalle el problema a resolver y los objetivos del proyecto son específicos y alcanzables.	Presenta el problema y los objetivos, pero carecen de claridad o no están completamente alineados.	Los objetivos son vagos o incompletos y no se relacionan adecuadamente con el problema a resolver.	No se presenta una descripción del problema ni los objetivos.	
Diseño y Metodología	Detalla las herramientas y técnicas de Python a emplear y justifica su uso para resolver el problema.	Enumera las herramientas y técnicas de Python, pero la explicación de su uso es incompleta o superficial.	Las herramientas o técnicas no están especificadas correctamente o no se relacionan con el problema.	No se describe herramientas ni técnicas de Python.	
Cronograma y Resultados	Presenta un cronograma detallado con hitos clave y resultados esperados bien definidos y relevantes al problema.	Incluye un cronograma y resultados esperados, pero no son suficientemente detallados o relevantes.	El cronograma y los resultados esperados son poco claros, incompletos o no están alineados con los objetivos.	No se presenta cronograma ni resultados esperados.	
Nota		Total, 9pts	Puntos oct.:	Calificación	

Proyecto 20%

Evaluación del Proyecto Final

Criterio	Excelente (5 puntos)	Bueno (3 puntos)	Insuficiente (1 punto)	No Presentado (0 puntos)	Puntaje Obtenido
Definición del problema y datos	Problema y datos claramente definidos, con relevancia explicada en un contexto práctico o teórico.	Problema y datos parcialmente definidos o relevancia no completamente clara.	Problema y datos vagos o sin definir, contexto ausente o no relevante.	No presenta definición del problema y datos	
Carga y Lectura de Archivos	Lectura y carga de archivos implementada correctamente, manejando errores y formatos diversos.	Lectura y carga implementada parcialmente, con errores menores o soporte limitado.	Lectura y carga de archivos incompleta o no implementada.	No realiza Carga y Lectura de Archivos	
Análisis Completo de Datos	Análisis completo, correcto y con resultados relevantes, incluyendo visualizaciones claras.	Análisis parcial y correcto, pero sin profundidad o visualizaciones limitadas.	Análisis incompleto, incorrecto o con resultados irrelevantes.	No presenta análisis Completo de Datos	
Metodología IteraFlex	Sigue todos los pasos de IteraFlex con claridad, asegurando un diseño funcional del proyecto.	Sigue parcialmente los pasos de IteraFlex. Implementación con deficiencias.	No sigue la metodología IteraFlex o la implementación es incorrecta.	No desarrolla metodología iteraflex	
Uso avanzado de variables	Emplea variables adecuadamente, optimizando su uso en el código.	Emplea variables parcialmente, con algunos errores o uso limitado.	No emplea variables adecuadamente o su uso es incorrecto.	No implementa uso avanzado de variables	
Uso de listas y diccionarios	Uso completo y eficaz de listas y diccionarios para resolver problemas.	Uso parcial o limitado de listas y diccionarios.	No emplea listas ni diccionarios o su uso es incorrecto.	No utiliza listas y diccionarios	
Conversión de datos	Realiza conversiones correctamente, sin errores.	Realiza conversiones parcialmente, con algunos errores menores.	Conversiones no realizadas o incorrectas.	No realiza conversión de datos	
Estructuras de control	Uso completo y correcto de ciclos y condicionales para resolver problemas.	Uso parcial o limitado de estructuras de control.	No utiliza ciclos ni condicionales o su implementación es incorrecta.	No implementa estructuras de control	

Criterio	Excelente (5 puntos)	Bueno (3 puntos)	Insuficiente (1 punto)	No Presentado (0 puntos)	Puntaje Obtenido
Análisis estadístico simple	Implementado correctamente, con resultados claros y relevantes.	Parcialmente implementado, resultados limitados o con algunos errores menores.	No implementado o resultados incorrectos.	No realiza análisis estadístico simple	
Interacción avanzada con usuario	Interacción significativa, con mensajes claros y funcionalidad completa.	Interacción parcial o funcionalidad básica.	Sin interacción significativa con el usuario.	No implementa interacción avanzada con usuario	
Comentarios en el código	Código bien documentado, con comentarios explicativos y claros.	Comentarios parciales, poco claros o ausentes en partes importantes del código.	Sin comentarios o documentación en el código.	No realiza comentarios en el código	
Exposición del Proyecto	Expone de manera clara, estructurada y siguiendo un flujo lógico. Demuestra dominio del tema y responde preguntas con seguridad.	Exposición con dificultades para seguir el flujo lógico. Responde preguntas con vacilación.	Exposición desorganizada o confusa. No demuestra dominio ni responde preguntas.	No realiza exposición del Proyecto	
Nota		Total, 60pts	Puntos oct.:	Calificación:	